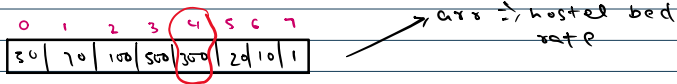


Arrays

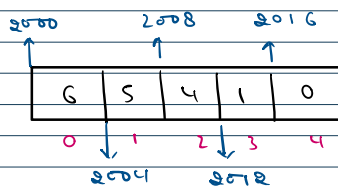
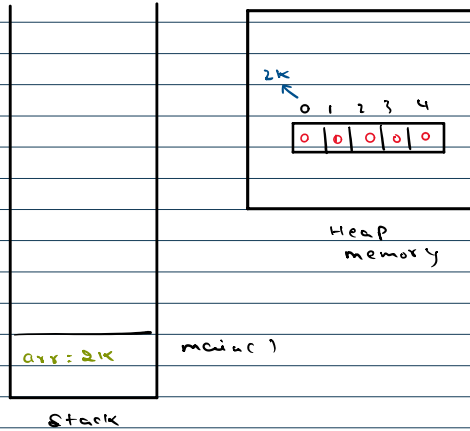
It is a non-primitive data structure of fixed size that stores multiple values of same data type in a single variable.

Each value in the array is accessed using an index, which starts from 0.



class → non-primitive → Heap memory

```
public class Main {  
    public static void main(String[] args) {  
        int[] arr = new int[5];  
    }  
}
```



$arr[2] \rightarrow 4$
↓
(base + 2 * (size of data type))
address
↓
 $2K + 2 * 4$
↓
 208

Base + index * size of
address element

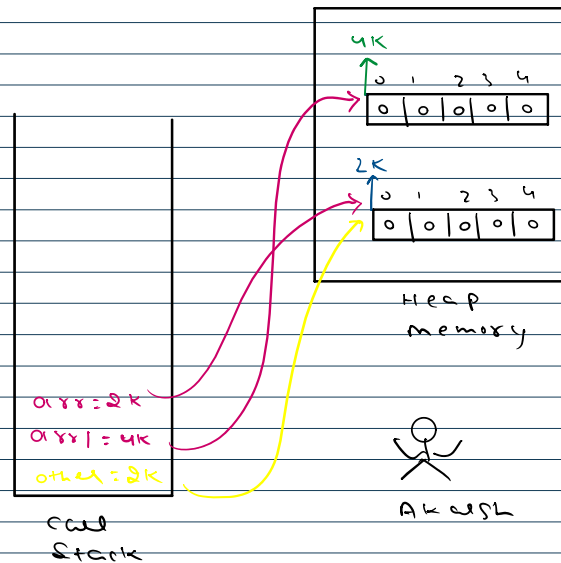
```
public class Main {  
    public static void main(String[] args) {  
        int[] arr = new int[5];  
        int arr1[] = new int[5]; // C Type  
        System.out.println(arr);  
        System.out.println(arr[0]);  
        System.out.println(arr[1]);  
        // System.out.println(arr[5]); // ArrayIndexOutOfBoundsException  
  
        int[] other = arr;  
        System.out.println(other);  
        System.out.println(other.length);  
    }  
}
```

Array operations

get $\Rightarrow$ `int x = arr[2];`

set $\Rightarrow$ `arr[2] = 100;`

```
1. public class Main {  
2.     public static void main(String[] args) {  
3.         int[] arr = new int[5];  
4.         int arr1[] = new int[5]; // C Type  
5.         System.out.println(arr);  
6.         System.out.println(arr[0]);  
7.         System.out.println(arr[1]);  
8.         // System.out.println(arr[5]); // ArrayIndexOutOfBoundsException  
9.  
10.        int[] other = arr;  
11.        System.out.println(other);  
12.  
13.    }  
14. }  
15.  
16. }
```



tOfBoundsException

Taking User Input

0	1	2	3	4
10	20	30	40	50

$i = 0$
 $i = 1$
 $i = 2$
 $i = 3$
 $i = 4$

$arr[0] = 10$
 $arr[1] = 20$
 $arr[2] = 30$
 $arr[3] = 40$
 $arr[4] = 50$

user Input

```

for(int i=0; i<5; i++) {
    arr[i] = sc.nextInt();
}

```

```

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i=0; i<arr.length; i++) {
            arr[i] = sc.nextInt();
        }

        display(arr);
    }

    public static void display(int[] arr) {
        for(int i=0; i<arr.length; i++) {
            System.out.print(arr[i] + " ");
        }
        System.out.println();
    }
}

```

ways to declare

```

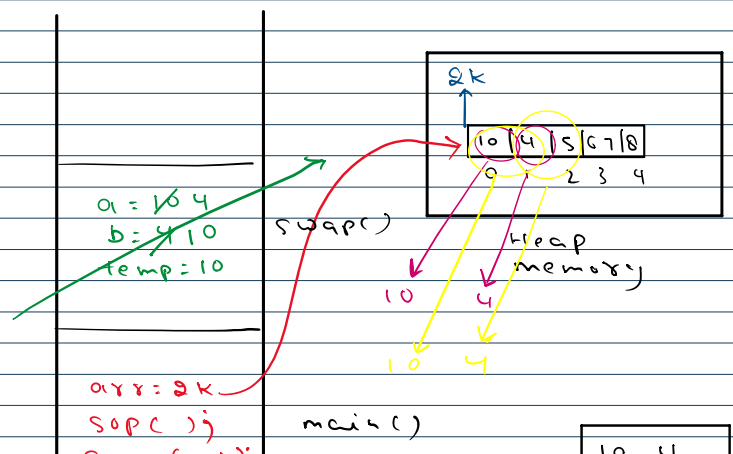
public class Main {
    public static void main(String[] args) {
        int[] arr1 = {10, 4, 5, 67, 8};
        int arr2[] = {10, 4, 5, 67, 8};
        int[] arr3 = new int[] {10, 4, 5, 67, 8};
        int arr4[] = new int[] {10, 4, 5, 67, 8};
        int[] arr5 = new int[5];
        int arr6[] = new int[5];
    }
}

```

```

1 public class Main {
2     public static void main(String[] args) {
3         int[] arr = {10, 4, 5, 67, 8};
4         System.out.println(arr[0] + " " + arr[1]);
5         swap(arr[0], arr[1]);
6         System.out.println(arr[0] + " " + arr[1]);
7     }
8
9     public static void swap(int a, int b) {
10        int temp = a;
11        a = b;
12        b = temp;
13    }
14
15 }

```





```

13 }
14
15 }

```

```

arr: 2K
SOPC();
swap(10, 4);
SOPC();

```

main()

10	4
10	4

cell
stack

console

```

public class Main {
    public static void main(String[] args) {
        int[] arr = {10, 4, 5, 67, 8};
        System.out.println(arr[0] + " " + arr[1]);
        swap(arr[0], arr[1]);
        System.out.println(arr[0] + " " + arr[1]);
    }

    public static void swap(int a, int b){
        int temp = a;
        a = b;
        b = temp;
    }
}

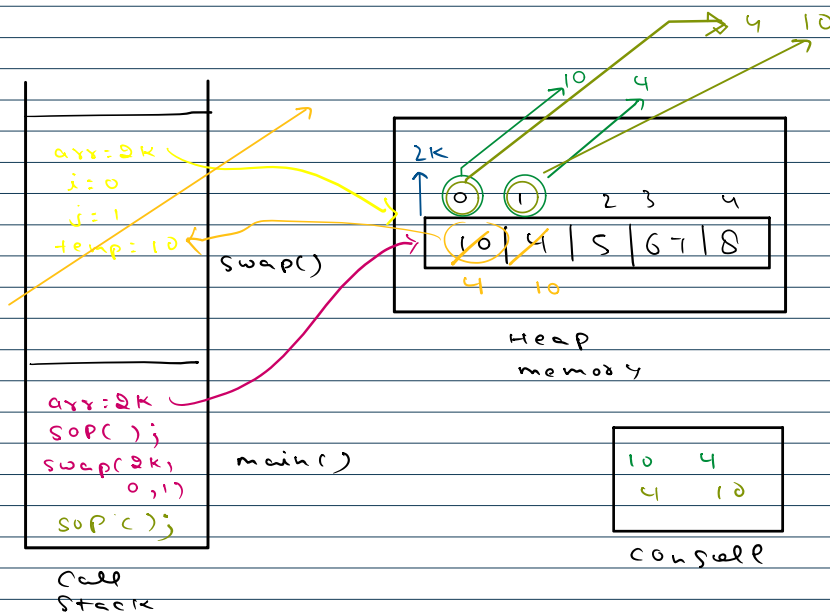
```

```

public class Main {
    public static void main(String[] args) {
        int[] arr = {10, 4, 5, 67, 8};
        System.out.println(arr[0] + " " + arr[1]);
        swap(arr, 0, 1);
        System.out.println(arr[0] + " " + arr[1]);
    }

    public static void swap(int[] arr, int i, int j){
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}

```



```

public class Main {
    public static void main(String[] args) {
        int[] arr = {10, 4, 5, 67, 8};
        System.out.println(arr[0] + " " + arr[1]);
        swap(arr, 0, 1);
        System.out.println(arr[0] + " " + arr[1]);
    }

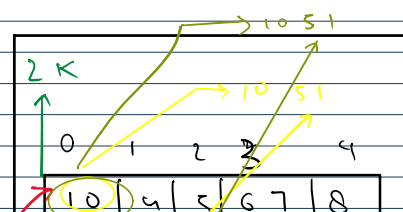
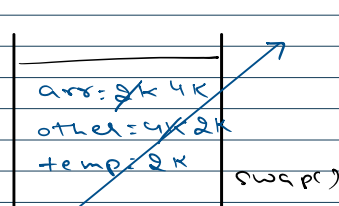
    public static void swap(int[] arr, int i, int j){
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}

```

```

1 public class Main {
2     public static void main(String[] args) {
3         int[] arr = {10, 4, 5, 67, 8};
4         int[] other = {31, 51, 11, 71, 81};
5         System.out.println(arr[0] + " " + other[1]);
6         swap(arr, other);
7         System.out.println(arr[0] + " " + other[1]);
8     }
}

```





```

5 System.out.println(arr[0] + " " + other[1]);
6 swap(arr, other);
7 System.out.println(arr[0] + " " + other[1]);
8 }
9
10 public static void swap(int[] arr, int[] other){
11     int[] temp = arr;
12     arr = other;
13     other = temp;
14 }
15 }

```

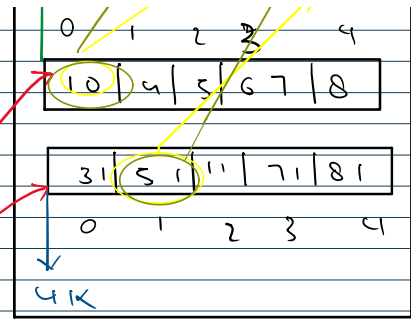
other = 4x2k
temp = 2k

arr = 2k
other = 4k
SOPC();
SOPC();

call
Stack

swap()

main()



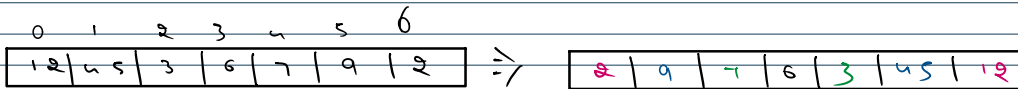
Heap
memory

10 5 1
10 5 1
console

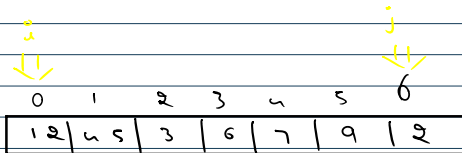
Reverse an array

arr → 12, 45, 3, 6, 7, 9, 2

↓
reverse → 2, 9, 7, 6, 3, 45, 12
of
arr



swap(arr[0], arr[6])
swap(arr[1], arr[5])
swap(arr[2], arr[4])



i: 0 → 1 → 2
j: 6 → 5 → 4

all
good ← i < j

swap(arr[0], arr[6])
swap(arr[1], arr[5])
swap(arr[2], arr[4])

i: 0, j: 6
i: 1, j: 5
i: 2, j: 4
i: 3, j: 3 → Stop

i >= j

Reverse part of an array

arr → 1, 5, 2, 8, 3, 11, 24, 73, 4, 6, 5, 84

Input → arr
i = 6
j = 9

i: 6



arr → 1, 5, 2, 8, 3, 11, 24, 73, 4, 6, 5, 84

$i = 0$
 $j = 9$

1, 5, 2, 8, 3, 11, 6, 4, 73, 24, 5, 84

all good ← $j < j$

$j++$ (swap(arr[0], arr[9])
 $j--$ (swap(arr[7], arr[8])
 $i++$ (swap(arr[8], arr[7])
 $j--$

Find maximum in an array

arr → 2, 3, 5, 4, 7, 2

max = -∞

2 ≥ 2 > max ⇒ ✓ arr[0]
 3 ≥ 3 > max ⇒ ✓ arr[1]
 5 ≥ 5 > max ⇒ ✓ arr[2]
 4 ≥ 4 > max ⇒ ✗ arr[3]
 7 ≥ 7 > max ⇒ ✓ arr[4]
 2 ≥ 2 > max ⇒ ✗ arr[5]

smallest value in java
↓
Integer.MIN_VALUE

```

public class Main {
    public static void main(String[] args) {
        int[] arr = {10, 4, 5, 67, 8};
        System.out.println(maxValue1(arr));
        System.out.println(maxValue2(arr));

        System.out.println(Math.max(4, 5));
        // System.out.println(Math.max(4, 5, 89));
        System.out.println(Math.max(4, Math.max(5, 89)));
    }

    public static int maxValue1(int[] arr) {
        int max = Integer.MIN_VALUE;
        for(int i=0; i<arr.length; i++){
            if(arr[i] > max){
                max = arr[i];
            }
        }
        return max;
    }

    public static int maxValue2(int[] arr) {
        int max = Integer.MIN_VALUE;
        for(int i=0; i<arr.length; i++){
            max = Math.max(arr[i], max);
        }
        return max;
    }
}
  
```

